REPORTE TÉCNICO

GALAGA

Raspberry Pi Pico (RP2040) · Pantalla OLED SSD1306 · Joystick analógico

Johne Geremy Arana Ramírez

Arquitectura de Computadoras 2 – Ingeniero Henry Sontay

Fecha: 7 de noviembre de 2026

1. Introducción y objetivo

Este proyecto implementa un videojuego tipo arcade llamado GALAGA escrito íntegramente en lenguaje ensamblador ARM (Thumb) para el microcontrolador RP2040 de la Raspberry Pi Pico, sin usar SDK ni lenguajes de alto nivel: todo el acceso al hardware se hace escribiendo directamente en los registros mapeados en memoria. El jugador controla una nave con un joystick analógico y debe esquivar cuatro obstáculos que caen, con tres vidas y un puntaje que sube por cada obstáculo esquivado. La dificultad aumenta con el puntaje.

El objetivo académico es demostrar el manejo de: pila (Stack), preservación de registros según el estándar AAPCS, interrupciones y temporizador por hardware, conversión analógica-digital (ADC), comunicación I²C y una máquina de estados.

2. Hardware utilizado

Componente	Pin de la Pico	Pin físico	Función
OLED SDA	GP4 (I2C0 SDA)	6	Datos I²C
OLED SCL	GP5 (I2C0 SCL)	7	Reloj I²C
OLED VCC / Joystick +5V	3V3 (OUT)	36	Alimentación a 3,3 V
OLED / Joystick GND	GND	38	Referencia
Joystick VRx	GP26 (ADC0)	31	Eje analógico canal 0
Joystick VRy	GP27 (ADC1)	32	Eje analógico canal 1
Joystick SW	GP22	29	Botón (disponible, no usado para iniciar)
LED integrado	GP25	—	Diagnóstico de arranque / OLED

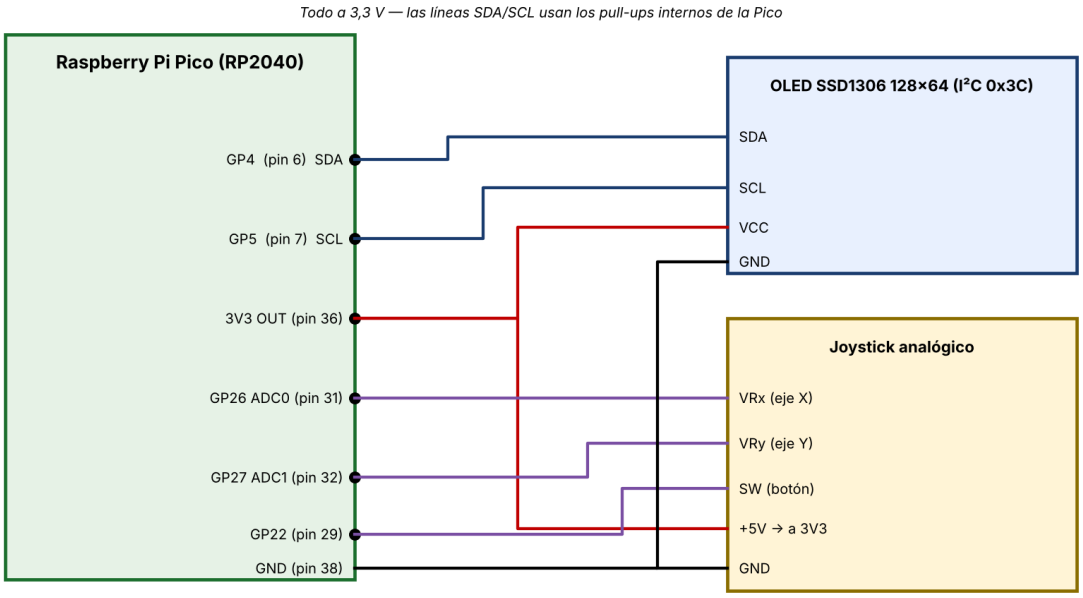


Figura 1. Esquema de conexión electrónica (todo a 3,3 V).

3. Arquitectura del software

El código está dividido en módulos independientes, cada uno con una responsabilidad:

Archivo	Responsabilidad
startup.s	Tabla de vectores, limpieza de .bss, VTOR, reloj (XOSC 12 MHz), tick de 1 µs y salida de reset de periféricos.
timer.s	Temporizador por hardware (ALARM0 → IRQ0 del NVIC). La ISR levanta frame_flag cada 33 333 µs y se re-arma sin acumular deriva.
inputs.s	Joystick: lectura de ADC, zona muerta, escala de velocidad, detección de "palanca movida" y espera de movimiento para iniciar.
video.s	I²C0, controlador SSD1306, framebuffer de 1 024 bytes, sprites 8×8, texto y números.
font.s	Fuente 5×7 (ASCII 32–90) en memoria de solo lectura.
game.s	Lógica del juego: nave, meteoros, colisión, vidas, puntaje, HUD, menú y Game Over.
led.s	LED GP25 y variable oled_status para diagnóstico.
main.s	Máquina de estados principal.
defs.inc	Constantes: direcciones base, offsets de registros y parámetros del juego.

3.1 Uso de la pila y estándar AAPCS

- **Registros:** r0–r3 y r12 son de uso libre (caller-saved) y llevan argumentos/retornos; r4–r7 son callee-saved: toda rutina que los usa los guarda con push al entrar y los restaura con pop al salir (por ejemplo game_update: push {r4-r6, lr} / pop {r4-r6, pc}).
- **Retorno:** las rutinas que llaman a otras guardan LR en la pila (push {lr}) y retornan con pop {pc}; las hojas (adc_read, i2c_write, led_*) solo usan r0–r3 y retornan con bx lr.

- **Interrupción:** timer_isr usa únicamente r0–r2; el hardware guarda automáticamente r0–r3, r12, LR, PC y xPSR en la pila al entrar a la excepción.
- **Excepción documentada:** el ayudante interno num_digit recibe sus parámetros en r4–r7 (convención interna) y es llamado solo desde fb_draw_number3/4, que sí guardan esos registros.
- **Pila:** el valor inicial de SP es 0x20042000 (tope de la SRAM) y crece hacia abajo; la profundidad máxima de llamadas es pequeña (main → game_draw → fb_draw_number3 → num_digit → fb_draw_char), muy por debajo del espacio libre.

4. Mapa de memoria

El programa se carga completo en la SRAM del RP2040 (264 KB, desde 0x20000000) mediante un archivo UF2; por eso, si se desconecta la alimentación, hay que cargarlo de nuevo. El script de enlace (link.ld) coloca las secciones así:

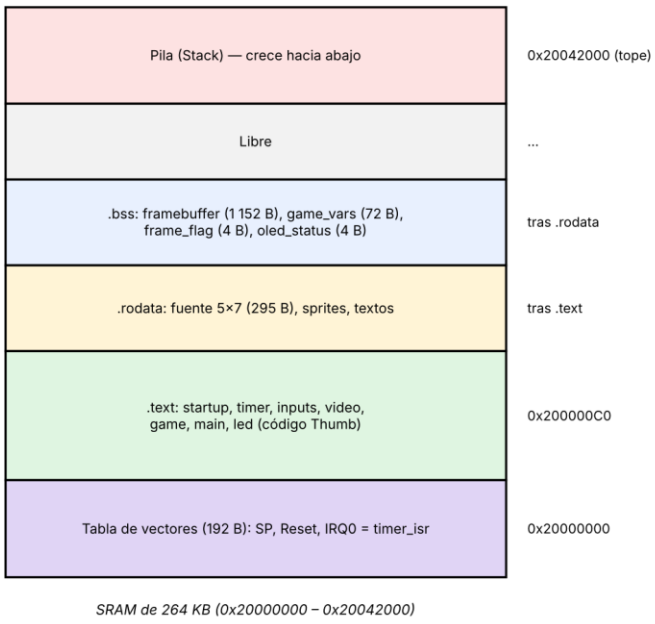


Figura 2. Mapa de memoria de GALAGA en SRAM.

Región	Dirección	Contenido
Tabla de vectores	0x20000000 (192 B)	SP inicial, Reset, IRQ0 = timer_isr, resto = default_handler
.text	0x200000C0 …	Código de todos los módulos (≈ 2 KB)
.rodata	a continuación de .text	Fuente 5×7 (295 B), sprites, cadenas de texto, secuencias de inicialización de la OLED
.bss	a continuación de .rodata	framebuffer (1 152 B = 1 024 + 128 de margen), game_vars (72 B), frame_flag (4 B), oled_status (4 B)

Región	Dirección	Contenido
Pila	0x20042000 hacia abajo	Registros guardados y direcciones de retorno

Nota: las direcciones exactas de cada símbolo se leen en el archivo build/galaga.map generado al compilar.

4.1 Estructura game_vars (72 bytes)

Campo	Offset	Descripción
G_PX / G_PY	0 / 4	Posición de la nave (x: 0–120, y: 8–56)
G_LIVES	8	Vidas restantes (inicia en 3)
G_SCORE	12	Puntaje
G_INV	16	Frames de invulnerabilidad tras un choque (45)
G_RNG	20	Estado del generador xorshift32 (semilla = TIMELR)
G_EX / G_EY / G_EV	24 / 40 / 56	x, y y velocidad de los 4 obstáculos (4 enteros de 32 bits cada uno)

5. Máquina de estados del juego

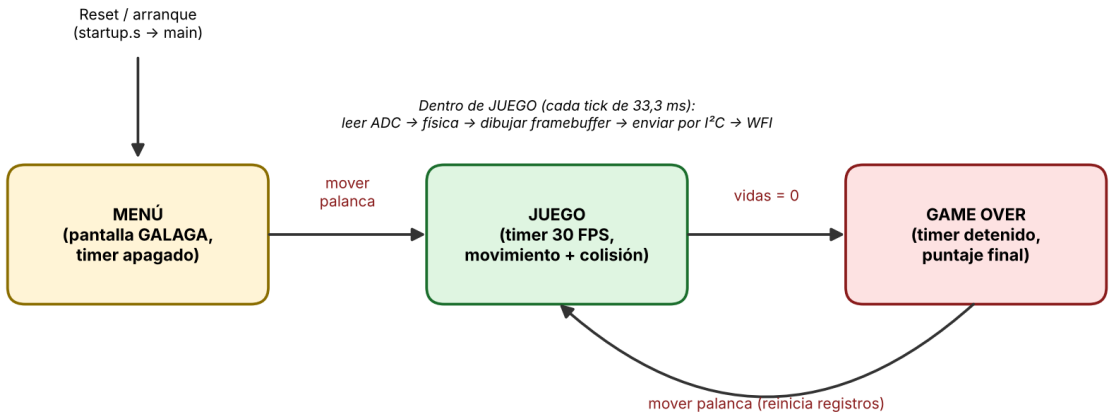


Figura 3. Diagrama de la máquina de estados (MENÚ → JUEGO → GAME OVER).

- **MENÚ:** se dibuja la pantalla de título GALAGA y se espera a que la palanca vuelva al centro y luego se mueva (input_wait_move). El timer permanece apagado.
- **JUEGO:** game_reset reinicia todos los registros de estado y timer_start arma la alarma. En cada tick de 33,3 ms se ejecutan game_update, game_draw y oled_flush.
- **GAME OVER:** al llegar a 0 vidas, timer_stop detiene totalmente el temporizador (se deshabilita la interrupción en el timer y en el NVIC), se muestra el puntaje y se espera de nuevo mover la palanca para reiniciar.

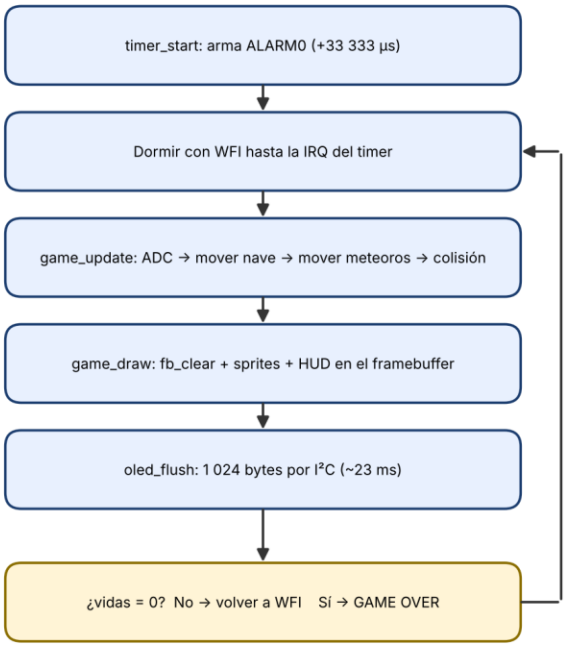


Figura 4. Lazo principal dentro del estado JUEGO.

6. Temporización y cálculo de tiempos de refresco

6.1 Reloj del sistema

El sistema usa el cristal externo de 12 MHz (XOSC) directamente como clk_ref y clk_sys, sin PLL. El bloque watchdog genera un tick de 1 µs dividiendo entre 12 (12 ciclos de 83,3 ns = 1 µs), que alimenta al TIMER de 64 bits.

6.2 Periodo de fotograma

FRAME_US = 33 333 µs ⇒ frecuencia = 1 / 33,333 ms ≈ 30,0 FPS. La alarma se re-arma sumando FRAME_US al valor anterior de ALARM0 (no al tiempo actual), de modo que el periodo es exacto y no acumula deriva.

6.3 Tiempo de transferencia al OLED por I²C

El bus I²C se configura en modo rápido (400 kHz): IC_FS_SCL_HCNT = 12 y IC_FS_SCL_LCNT = 18, es decir 30 ciclos de 12 MHz = 2,5 µs por bit ⇒ 400 kHz. Cada byte requiere 9 bits de reloj (8 de datos + 1 de ACK):

Cálculo	Resultado
Tiempo por byte = 9 / 400 kHz	22,5 µs
Ventana de dirección (comandos): 8 bytes (dirección + control + 6 comandos) × 22,5 µs	≈ 0,18 ms
Datos: dirección + control + 1 024 bytes = 1 026 bytes × 22,5 µs	≈ 23,1 ms

Cálculo	Resultado
Total por fotograma (oled_flush)	≈ 23,3 ms
Presupuesto por fotograma	33,3 ms
Holgura para ADC + física + dibujo	≈ 10 ms (≈ 30 %)

Como 23,3 ms < 33,3 ms, la transferencia completa cabe en un periodo y el lazo principal alcanza a dormir con WFI antes del siguiente tick. La pantalla se redibuja completa a 30 FPS; el panel SSD1306 además se refresca solo a partir de su memoria interna (GDDRAM), sin intervención del software.

6.4 Lectura del ADC

El ADC usa clk_adc = XOSC (12 MHz). Una conversión toma 96 ciclos de clk_adc ⇒ 96 / 12 MHz = 8 µs por muestra. En cada fotograma se leen dos canales en game_update (16 µs), más dos lecturas en input_moved solo mientras se espera iniciar. Resolución de 12 bits sobre 3,3 V: 3,3 / 4096 ≈ 0,8 mV por paso.

6.5 Velocidad de la nave

La lectura se centra en 2048 y se aplica una zona muerta de ±400 cuentas (≈ ±0,32 V). Fuera de ella el desplazamiento por fotograma es (lectura − 2048) >> 9, es decir de ±1 a ±4 píxeles por fotograma, equivalente a 30–120 píxeles por segundo.

7. Entrada: joystick

7.1 Inicio del juego moviendo la palanca

En lugar de pulsar un botón, el juego inicia cuando la palanca se empuja más de 1 000 cuentas respecto al centro (≈ 0,8 V) en cualquier eje (MOVE_THRESH en defs.inc). La rutina input_wait_move primero espera a que la palanca esté en reposo y después a que se mueva, para evitar que se inicie o reinicie accidentalmente mientras aún se está empujando (por ejemplo, justo después de perder).

7.2 Corrección de ejes (joystick girado)

El módulo del joystick está montado girado 90°, por lo que al empujar a la derecha la nave bajaba. Se corrigió por software con tres constantes en defs.inc, sin modificar la lógica del juego:

Constante	Valor	Efecto
SWAP_XY	1	El eje horizontal de la nave lee el canal ADC1 y el vertical lee el canal ADC0.
INVERT_X	0	Sentido horizontal sin invertir.
INVERT_Y	1	Sentido vertical invertido (físicamente arriba = nave arriba).

8. Compilación y carga

Herramientas: Arm GNU Toolchain (arm-none-eabi-as, ld, objcopy) y Python 3.

```
arm-none-eabi-as -mcpu=cortex-m0plus -mthumb -I src -g src/<modulo>.s -o build/<modulo>.o
arm-none-eabi-ld -T link.ld -Map build/galaga.map -o build/galaga.elf <objetos>
arm-none-eabi-objcopy -O binary build/galaga.elf build/galaga.bin
python tools/bin2uf2.py build/galaga.bin build/galaga.uf2
```

El script build.bat (Windows) o make ejecuta estos pasos. Para cargar: mantener BOOTSEL pulsado al conectar la Pico por USB y copiar galaga.uf2 a la unidad RPI-RP2.

Diagnóstico con el LED integrado (GP25)

LED	Significado
Apagado siempre	El programa no arrancó (repetir carga con BOOTSEL).
Encendido fijo	Arrancó pero se detuvo antes del menú.
Parpadeo lento (~1 Hz)	Menú listo y la OLED respondió por I²C.
Parpadeo rápido (~8 Hz)	La OLED no respondió (revisar SDA, SCL, VCC, GND).

9. Conclusiones

- Se construyó un videojuego completo sin SDK, controlando directamente periféricos del RP2040 (reloj, timer, NVIC, ADC, I²C).
- El temporizador por hardware con interrupción y WFI permite un periodo exacto de 30 FPS sin ciclos de espera por software.
- La transferencia I²C (≈ 23,3 ms) cabe en el periodo de 33,3 ms, dejando ≈ 10 ms para la lógica.
- La organización modular y el uso disciplinado de la pila y de los registros callee-saved facilitan la lectura, la depuración y el mantenimiento.